

BỘ MÔN KHOA HỌC DỮ LIỆU

TRÍ TUỆ NHÂN TẠO

LAB 02 & 03 – ARTIFICIAL INTELLIGENCE



TS. TRẦN NGỌC VIỆT



HỌC KỲ 1 – NĂM HỌC 2024-2025



KHÓA K28 & K29

#THUẬT TOÁN TÌM KIẾM MÙ

**Chiến lược tìm kiếm mù (Uninformed search strategies)*

Chiến lược tìm kiếm mù chỉ sử dụng thông tin có sẵn được định nghĩa trong bài toán.

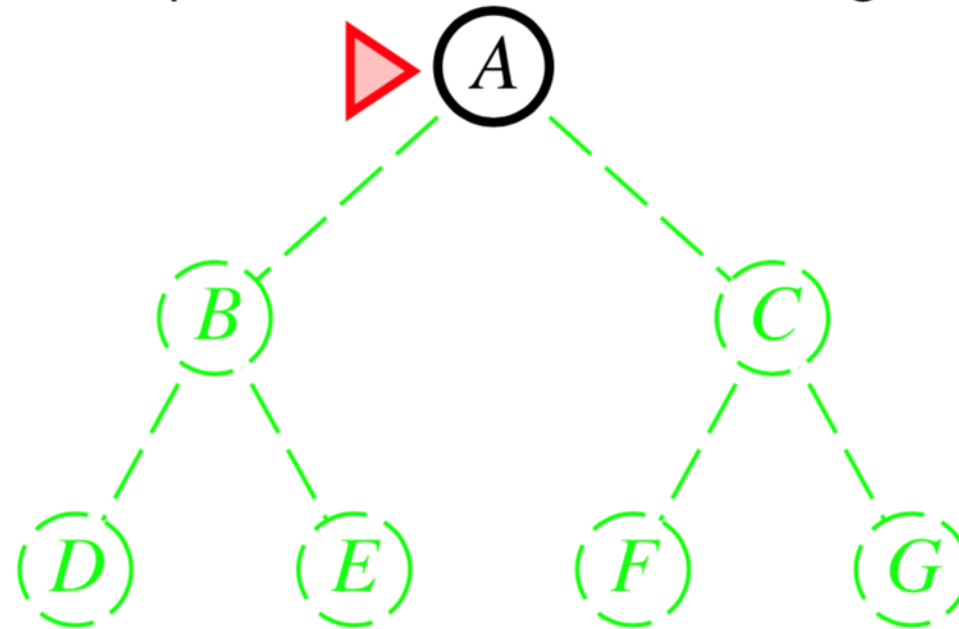
Các thuật toán tìm kiếm mù:

- Tìm kiếm theo chiều rộng (Breadth-first search)
- Tìm kiếm theo chiều sâu (Depth-first search)

#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

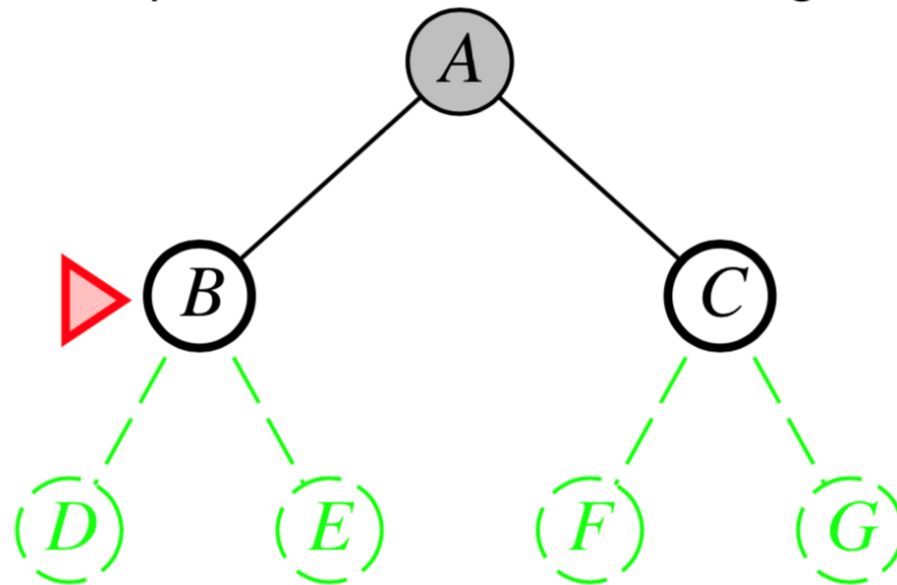
Mở rộng bằng hình thức vét cạn, sử dụng hàng đợi FIFO



#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

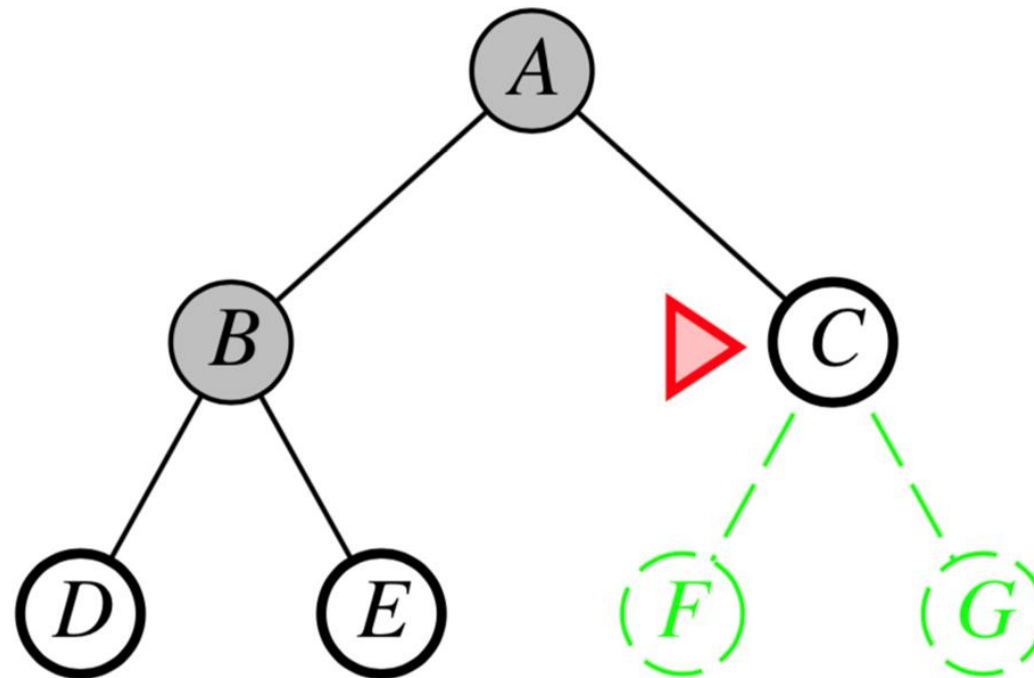
Mở rộng bằng hình thức vét cạn, sử dụng hàng đợi FIFO



#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

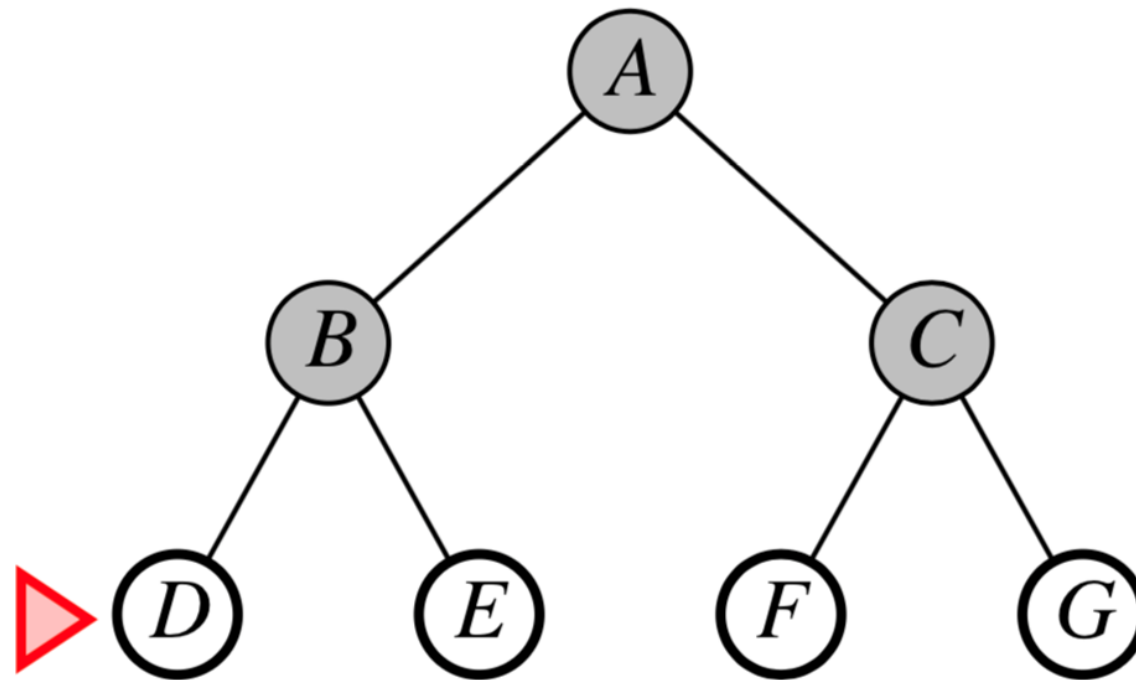
Mở rộng bằng hình thức vét cạn, sử dụng hàng đợi FIFO



#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

Mở rộng bằng hình thức vét cạn, sử dụng hàng đợi FIFO



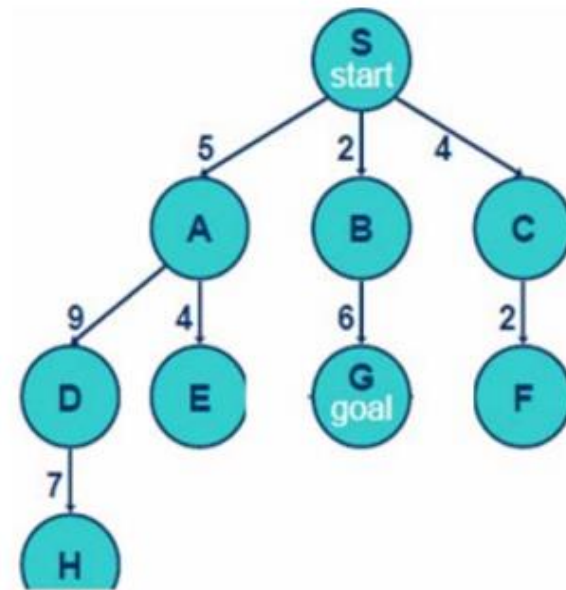
#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

Mở rộng bằng hình thức vét cạn, sử dụng hàng đợi FIFO

node	Queue	Father
	S	
S	A, B, C	Father[A,B,C]=S
A	B, C, D, E	Father[D,E]=A
B	C,D,E,G	Father[G]=B
C	D, E, G, F	Father[F]=C
D	E,G, F, H	Father[H]=D
E	G, F, H	
G	F, H	

Giá trị các biến trong
giải thuật theo chiều rộng



Cây tìm kiếm của giải thuật theo chiều rộng

#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

```
from collections import defaultdict
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def BFS(self, u):
        visited = [False] * (len(self.graph))

        queue = []

        visited[u] = True
        queue.append(u)

        while queue:
            u = queue.pop(0)
            print(u, end=' ')

            for i in self.graph[u]:
                if not visited[i]:
                    queue.append(i)
                    visited[i] = True
```

```
if __name__ == '__main__':
    g = Graph()
    g.addEdge(0, 1)
    g.addEdge(1, 0)
    g.addEdge(0, 2)
    g.addEdge(2, 0)
    g.addEdge(2, 4)
    g.addEdge(4, 2)
    g.addEdge(1, 4)
    g.addEdge(4, 1)
    g.addEdge(1, 3)
    g.addEdge(3, 1)
    g.addEdge(3, 4)
    g.addEdge(4, 3)
    g.addEdge(3, 5)
    g.addEdge(5, 3)
    g.addEdge(5, 4)
    g.addEdge(4, 5)

    print("BFS - duyệt tìm kiếm chiều rộng bắt đầu từ đỉnh 0")
    g.BFS(0)
```


#THỰC HÀNH 01

#Tìm kiếm theo chiều rộng

BFS - duyệt tìm kiếm chiều rộng bắt đầu từ đỉnh 0

0 1 2 4 3 5

#Nhận xét: BFS và vẽ đồ thị

#THỰC HÀNH 02

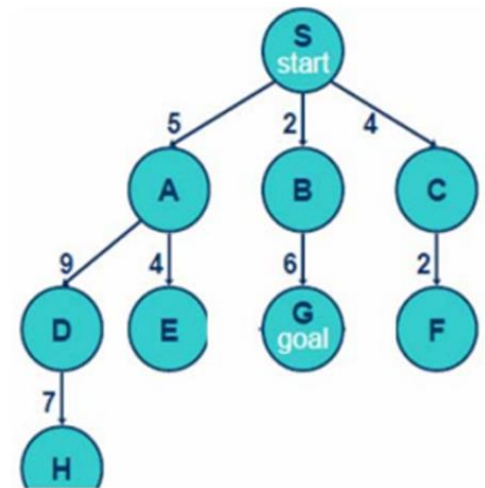
#Tìm kiếm theo chiều rộng

BFS - duyệt tìm kiếm chiều rộng bắt đầu từ đỉnh 0

0 1 2 3 4 5 6 7 8

node	Queue	Father
	S	
S	A, B, C	Father[A,B,C]=S
A	B, C, D, E	Father[D,E]=A
B	C,D,E,G	Father[G]=B
C	D, E, G, F	Father[F]=C
D	E,G, F, H	Father[H]=D
E	G, F, H	
G	F, H	

Giá trị các biến trong
giải thuật theo chiều rộng



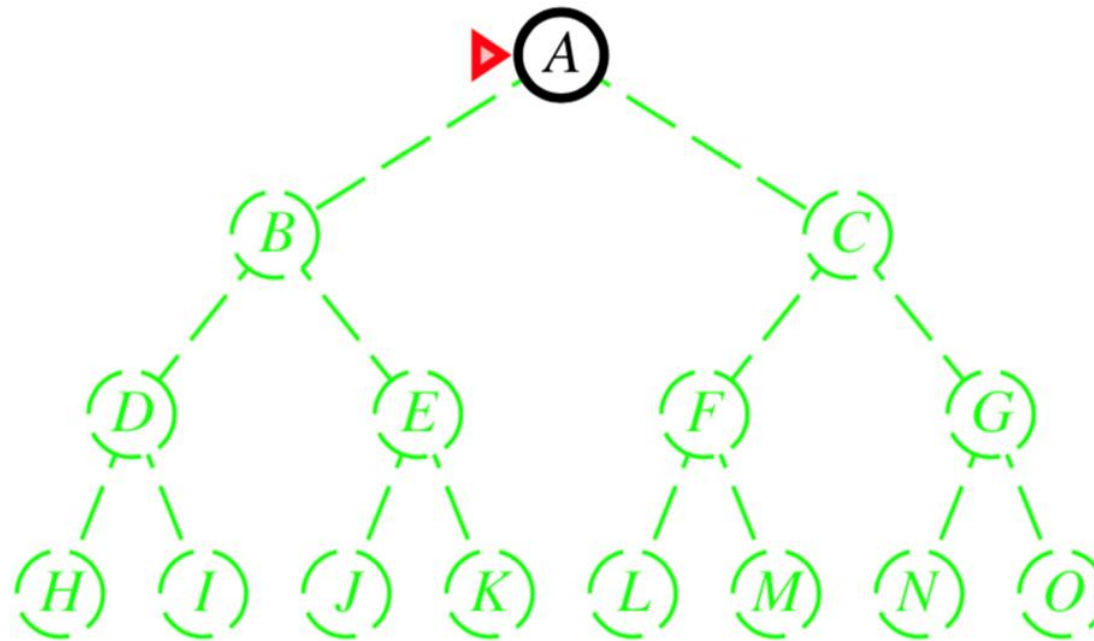
Cây tìm kiếm của giải thuật theo chiều rộng

#Nhận xét: BFS và vẽ đồ thị

#THỰC HÀNH 03

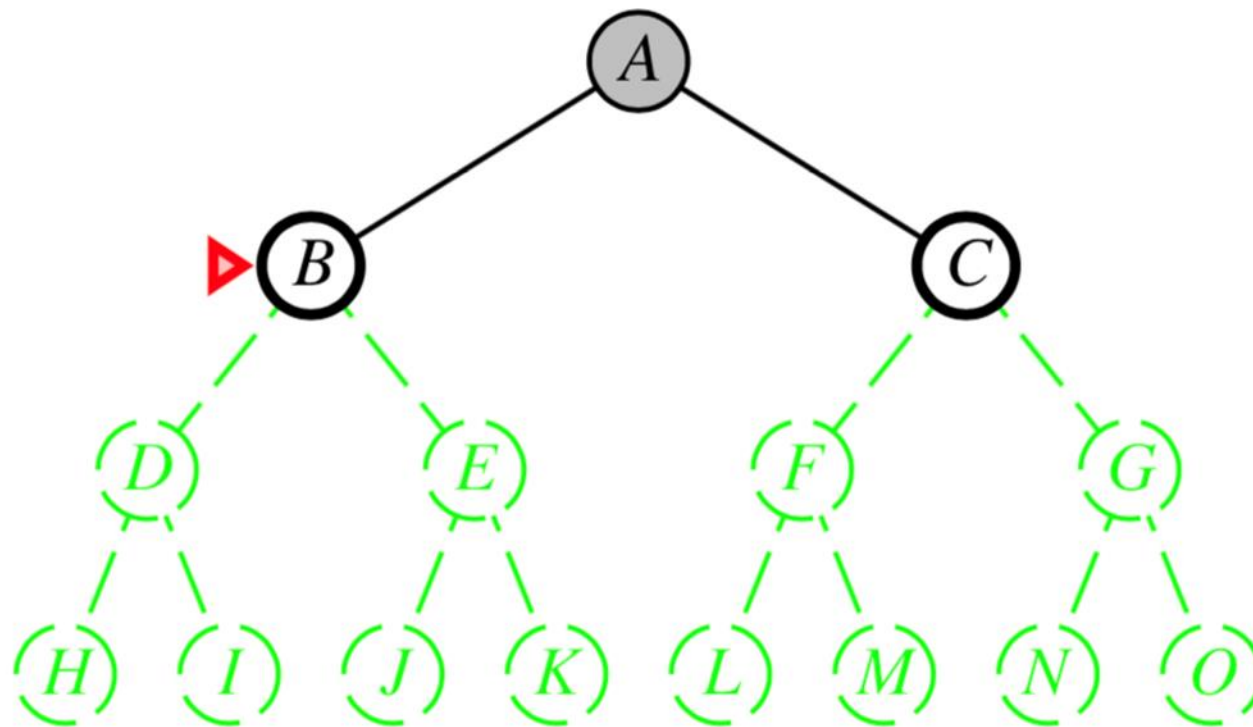
#Tìm kiếm theo chiều sâu

Sử dụng cấu trúc dữ liệu ngăn xếp (Stack) LIFO để lưu giữ các trạng thái lá của cây tìm kiếm



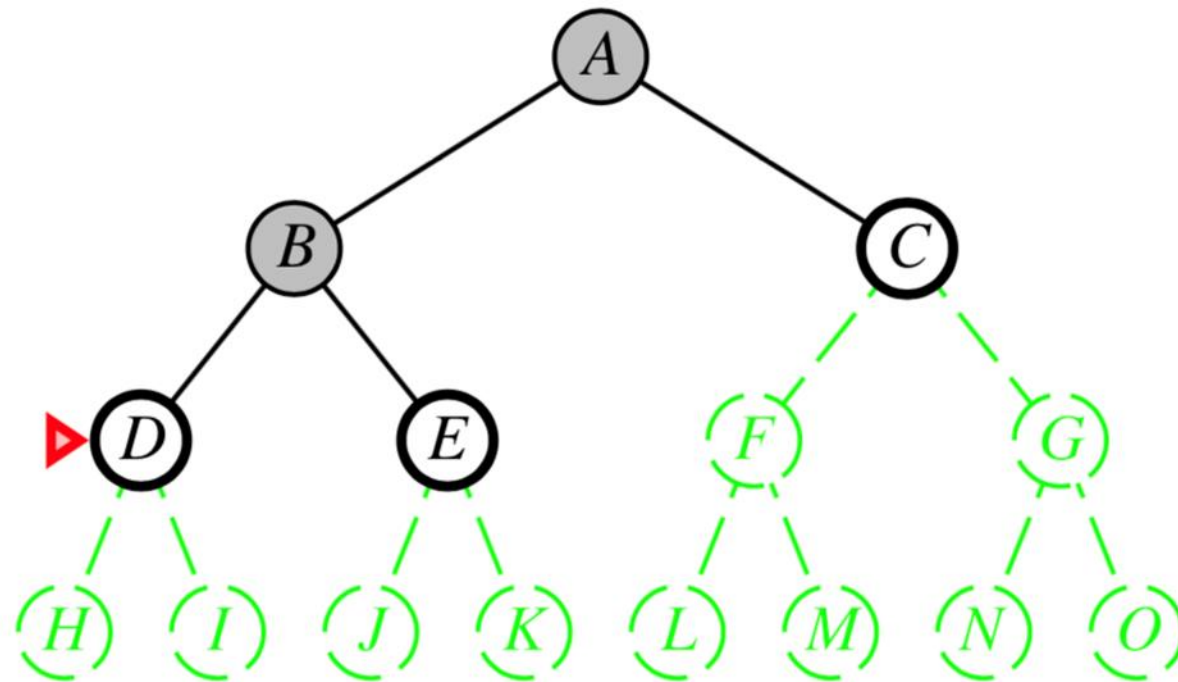
#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu



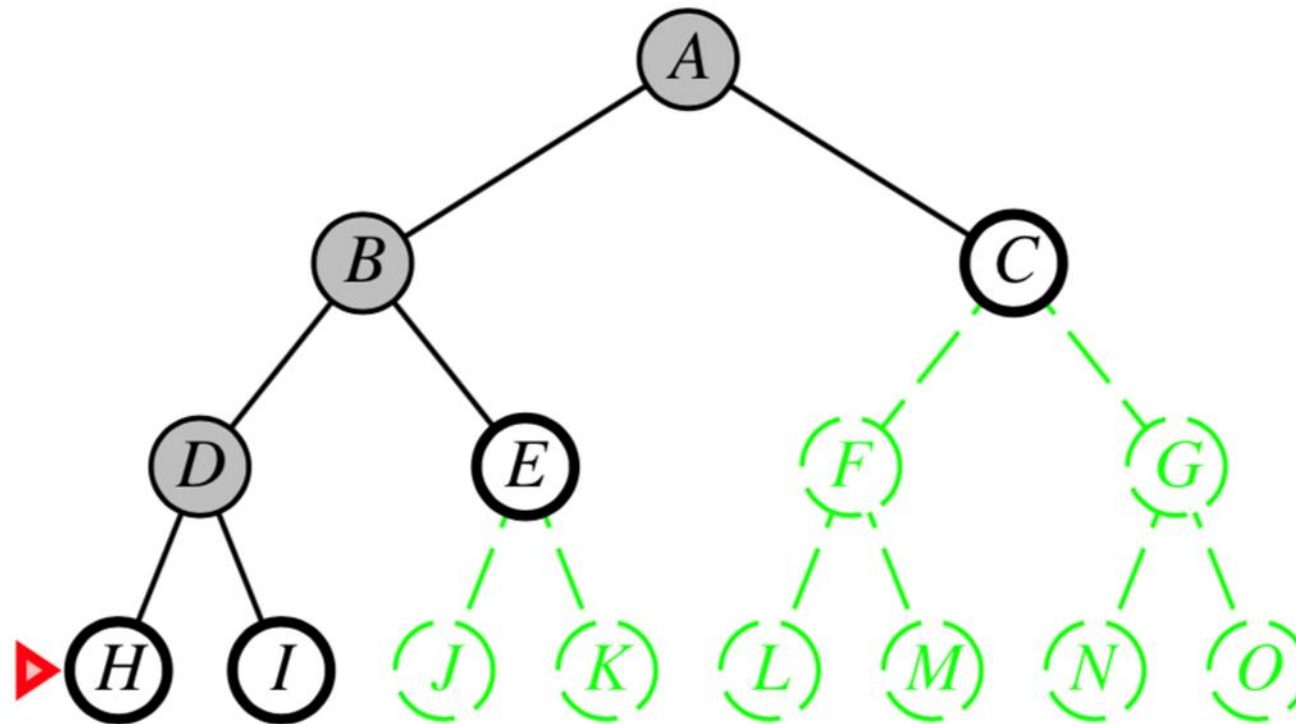
#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu



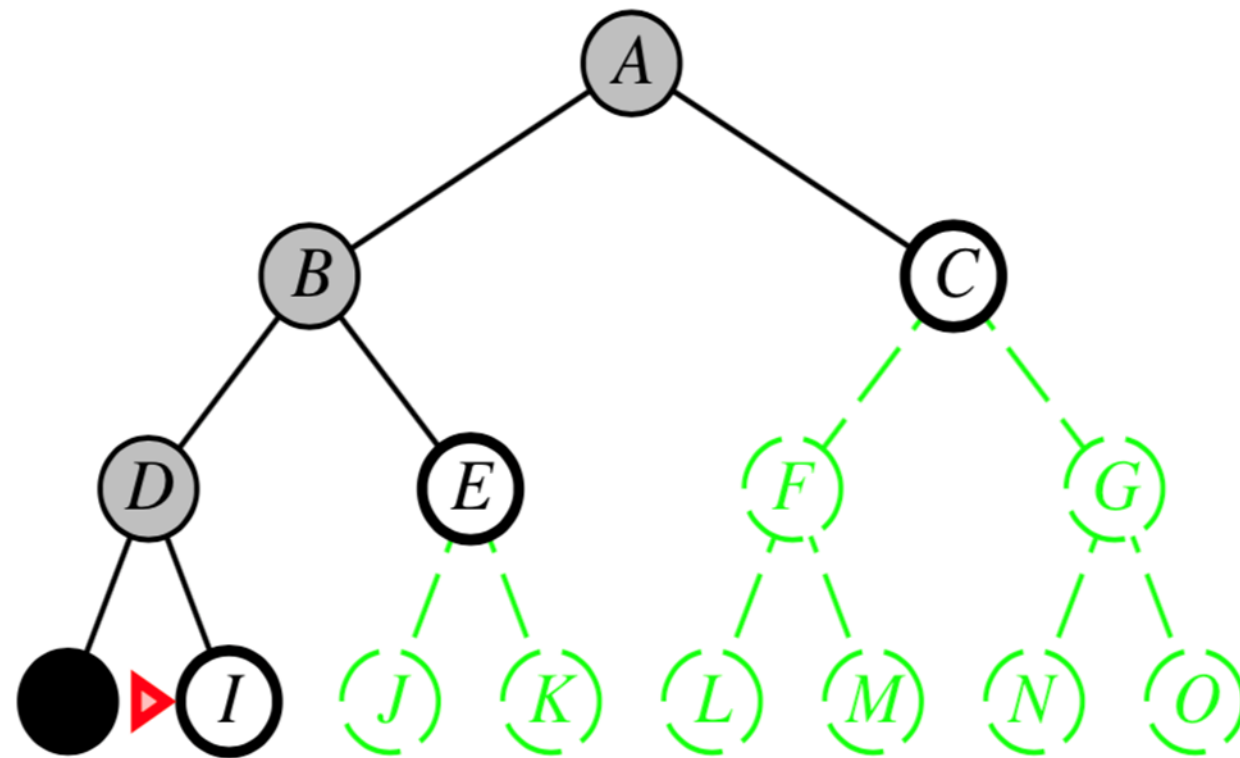
#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu



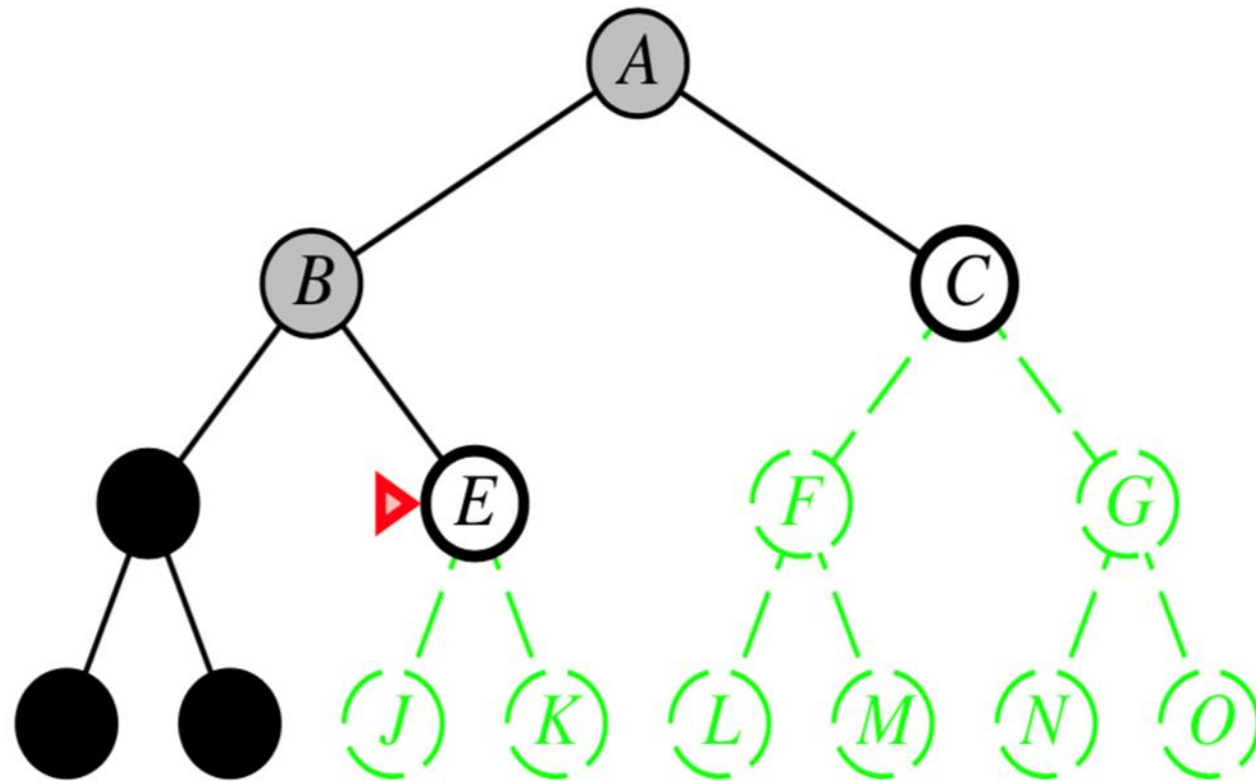
#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu



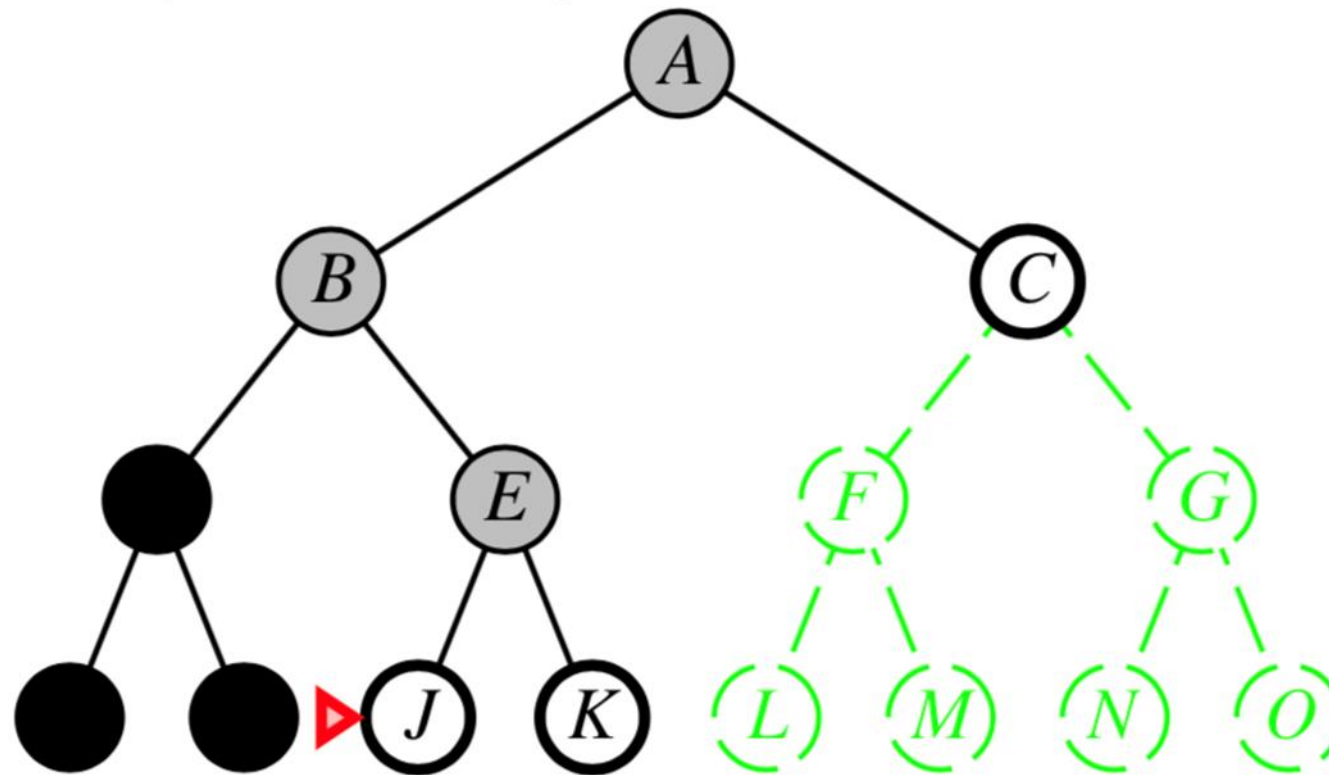
#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu



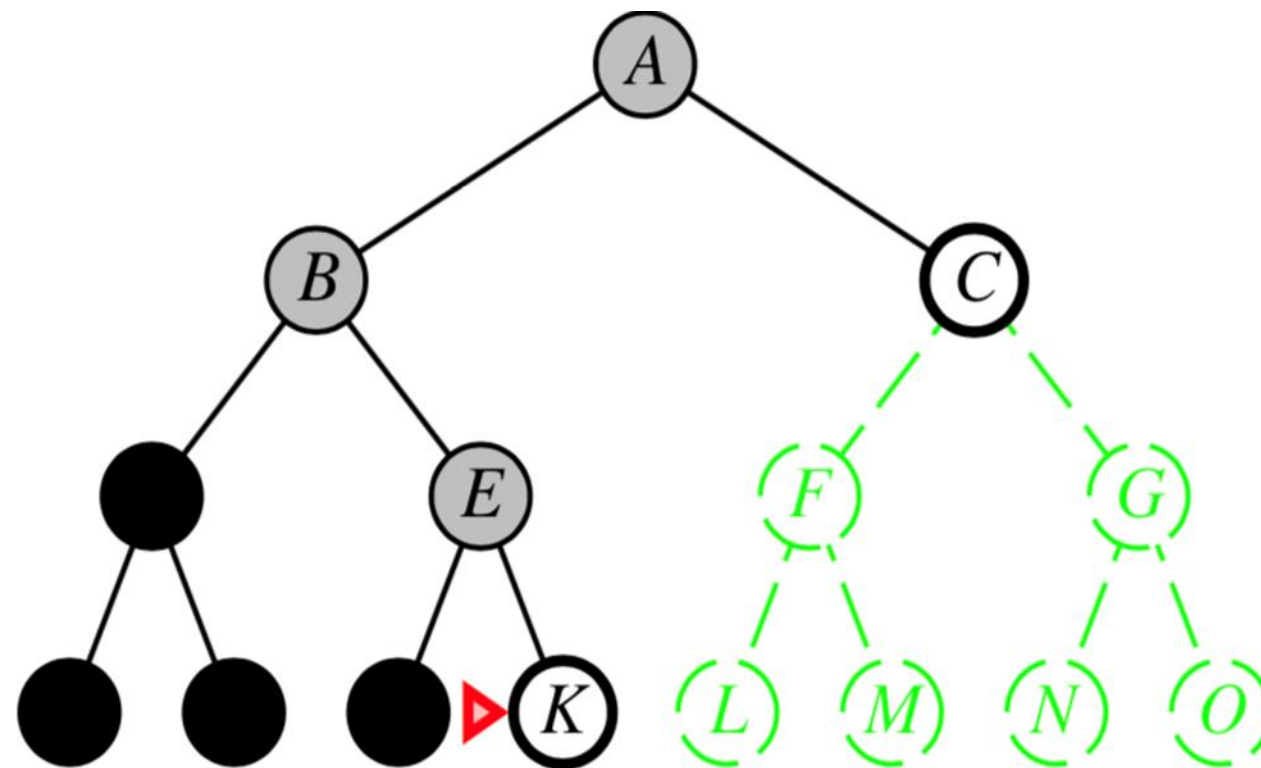
#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu



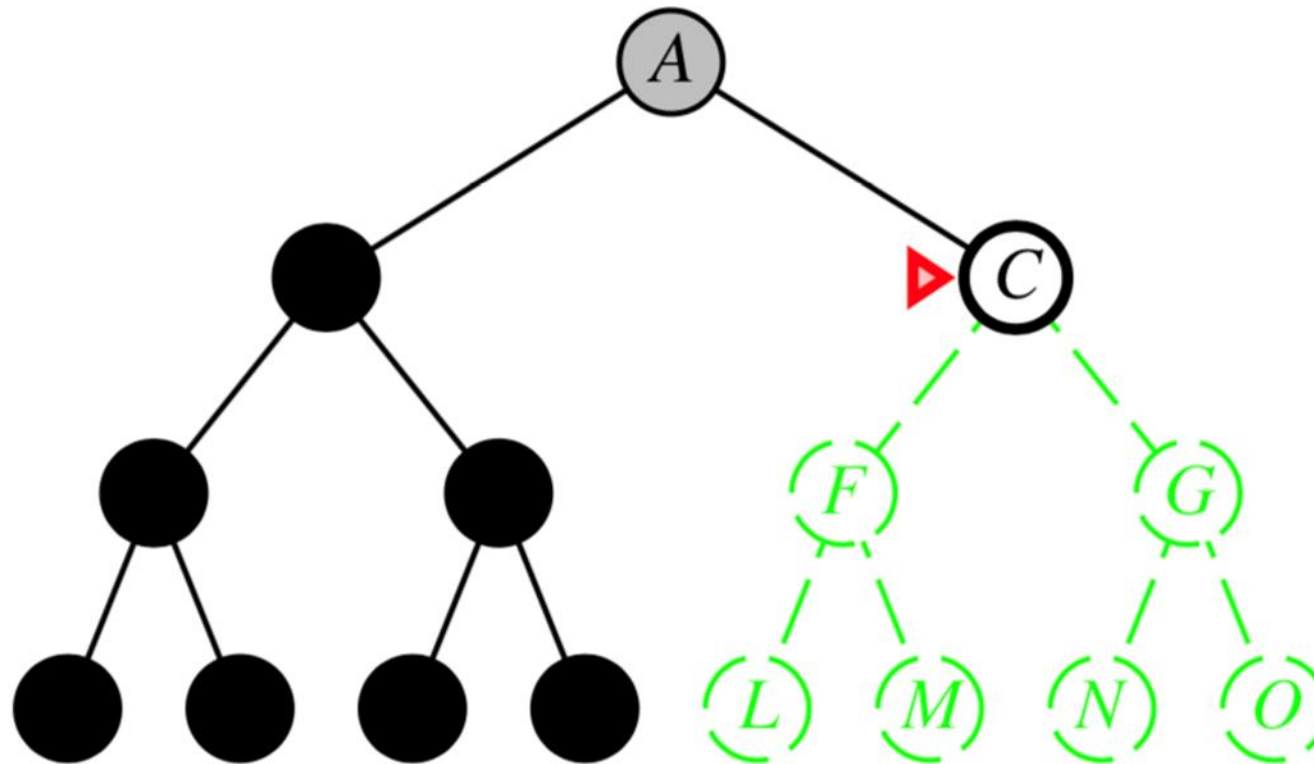
#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu



#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu

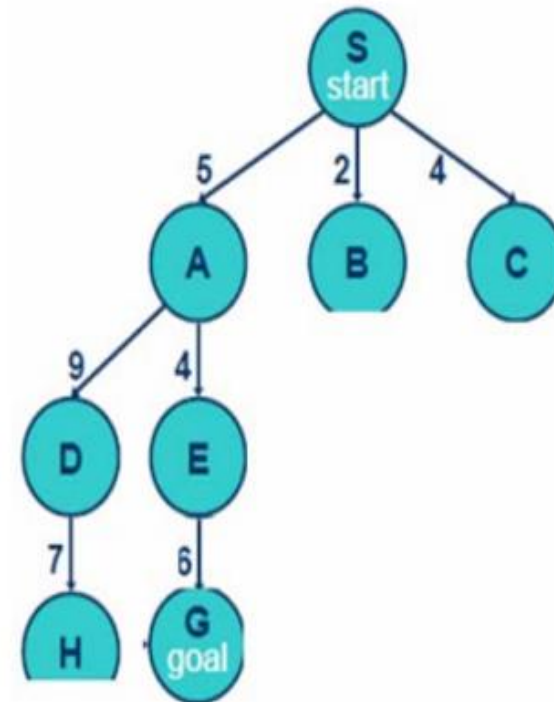


#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu

node	Stack	father
	S	
S	A, B, C	Father[A,B,C]=S
A	D, E, B, C	Father[D,E]=A
D	H, E, B, C	Father[H]=D
H	E, B, C	
E	G, B, C	Father[G]=E
G		

Giá trị các biến trong
giải thuật theo chiều sâu



Cây tìm kiếm của giải thuật theo chiều

#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu

```
from collections import defaultdict
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def DFSUtil(self, v, visited):
        visited[v] = True
        print(v, end=" ")

        for i in self.graph[v]:
            if visited[i] == False:
                self.DFSUtil(i, visited)

    def DFS(self, v):
        visited = [False] * (max(self.graph) + 1)

        self.DFSUtil(v, visited)
```

```
if __name__ == '__main__':
    g = Graph()
    g.addEdge(0, 1)
    g.addEdge(1, 0)
    g.addEdge(0, 2)
    g.addEdge(2, 0)
    g.addEdge(2, 4)
    g.addEdge(4, 2)
    g.addEdge(1, 4)
    g.addEdge(4, 1)
    g.addEdge(1, 3)
    g.addEdge(3, 1)
    g.addEdge(3, 4)
    g.addEdge(4, 3)
    g.addEdge(3, 5)
    g.addEdge(5, 3)
    g.addEdge(5, 4)
    g.addEdge(4, 5)
    print("DFS - Duyệt tìm kiếm từ vertex 0")
    g.DFS(0)
```

#THỰC HÀNH 03

#Tìm kiếm theo chiều sâu

DFS - Duyệt tìm kiếm từ vertex 0

0 1 4 2 3 5

#Nhận xét: DFS và vẽ đồ thị

#THỰC HÀNH 04

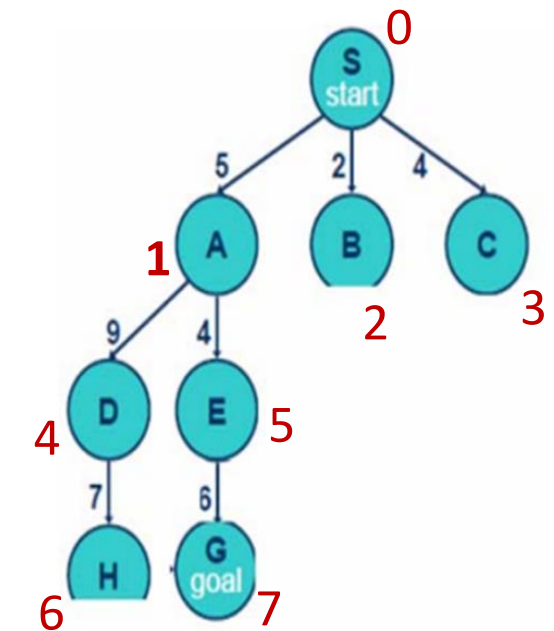
#Tìm kiếm theo chiều sâu

DFS - Duyệt tìm kiếm từ vertex 0

0 1 4 6 5 7 2 3

node	Stack	father
	S	
S	A, B, C	Father[A,B,C]=S
A	D, E, B, C	Father[D,E]=A
D	H, E, B, C	Father[H]=D
H	E, B, C	
E	G, B, C	Father[G]=E
G		

Giá trị các biến trong
giải thuật theo chiều sâu



Cây tìm kiếm của giải thuật theo chiều

#Nhận xét: DFS và vẽ đồ thị



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

